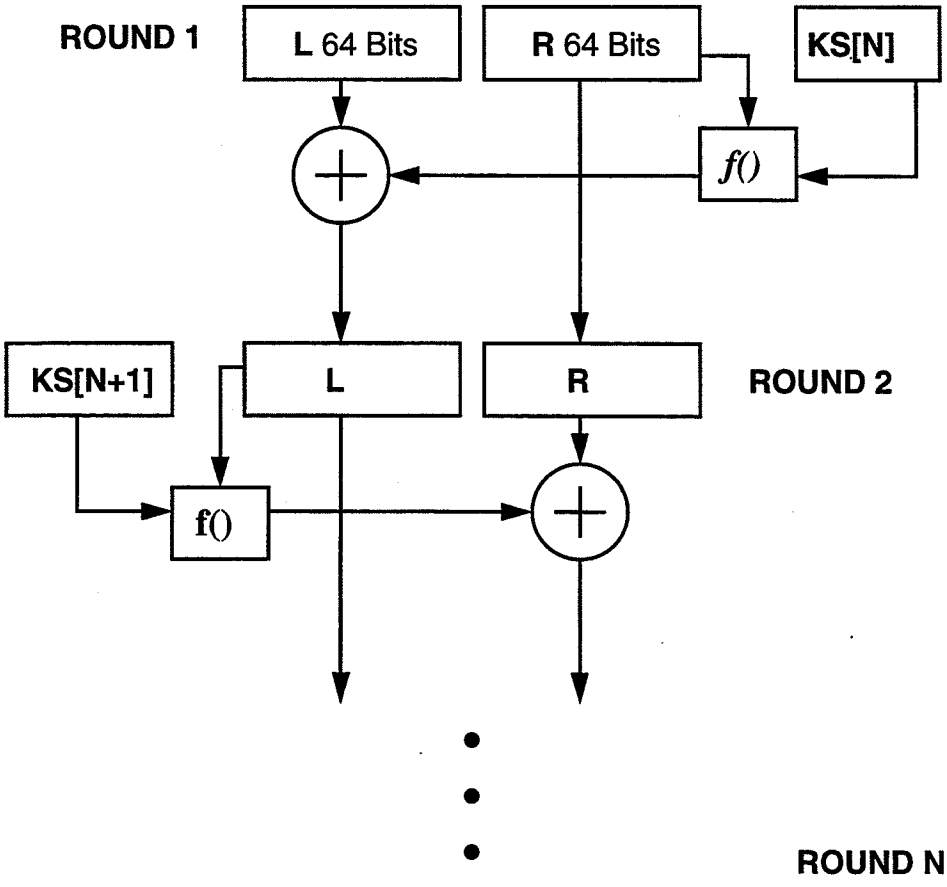
# *CRISP*: A Feistel cipher with hardened key-scheduling

*Marcus Leech, Nortel Technologies*

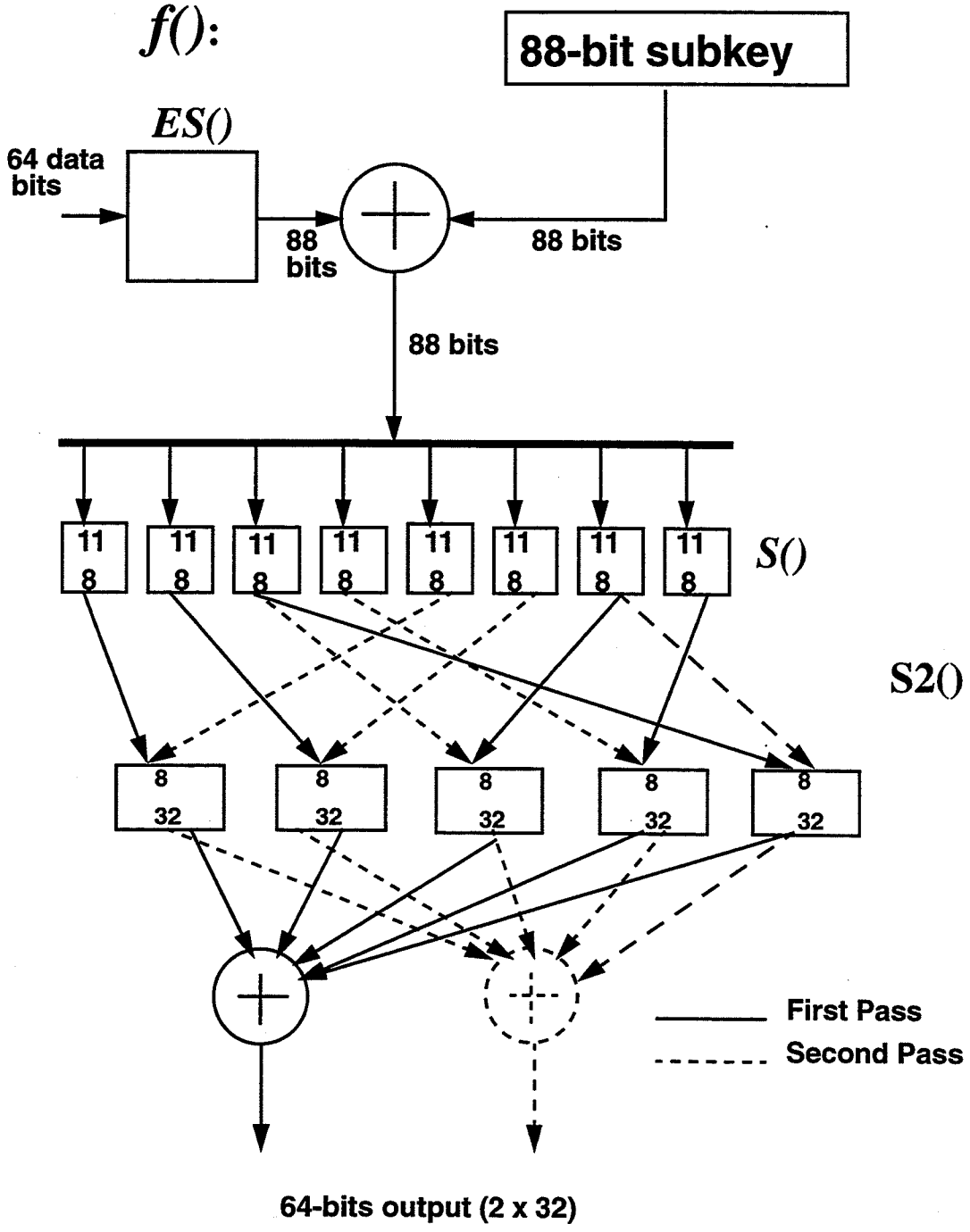
## Algorithm

This paper describes a new Feistel block cipher, *CRISP*, that uses itself as a PRNG in the key-scheduling function. The cipher consists of 6 rounds in which the left and right half input blocks are alternately modulo-2 added to a non-linear function of the other half input block, and the current key schedule bits.

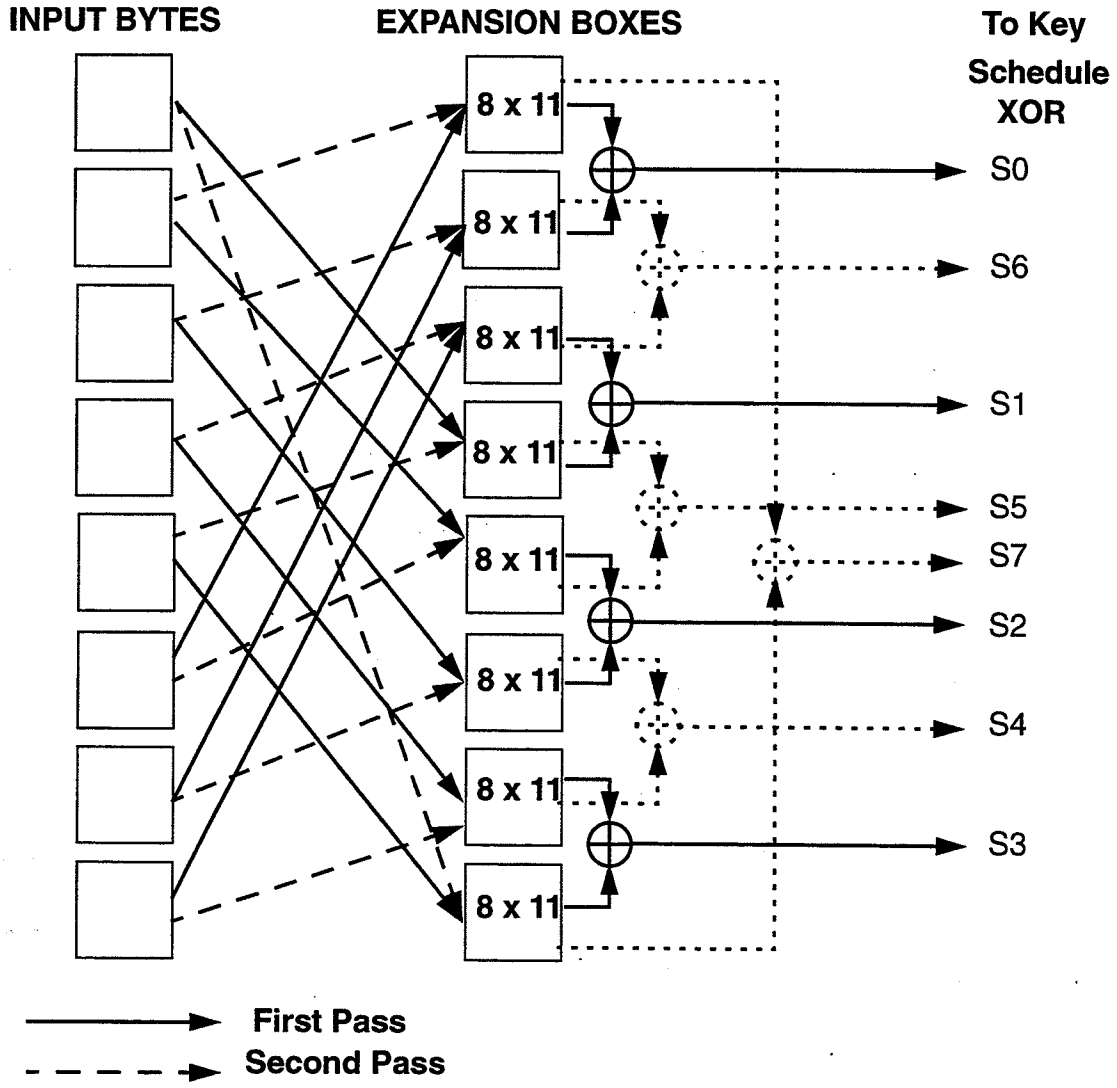



The cipher uses a 128-bit key, with a 128-bit data block. The non-linear round function, *f()*, computes a permute-substitute function of the current 88-bit key-schedule bits with the 64 input bits. The 64-bit input is first expanded to 88-bits using XOR-combined 8x11 bit S-boxes. The result of the expansion is then XORed with the 88 key bits, and fed through eight 11x8 S-boxes. The output of the S-boxes is then processed through *S2()*, a function that uses five 8x32 S-boxes that are

generated in a key-dependant way.



The *ES() expansion* function is defined as follows:




Example values for the *ES()* function tables are listed in Appendix A.

## S-Box design methodology

The primary S-boxes in *CRISP* are constructed according to design principals described in an article on S-box design by Gordon and Retkin[1]. In that article, they make the claim that the probability of linearity of an S-box is proportional to the inverse of the factorial of its size. Each S-box is based on a composition of eight 8x8 reversible, randomly permuted S-boxes. That is, for an 11-bit input, the low-order 8 bits select an 8-bit value from an 8x8 S-box, while the high-order 3 bits select which 8x8 S-box to use. This is identical to the structure used in the DES S-boxes.

The primary S-boxes are generated using a C program designed to find S-boxes with a given threshold pairs-XOR count and threshold linearity; the program generates random S-boxes, measures the maximum pairs-XOR count, and linearity and discards any S-boxes whose pairs-XOR count is above the threshold, or whose linearity is above the threshold. Linearity is computed by

measuring the hamming distance between all possible output vectors (combined under XOR) against all linear-boolean function vectors of the input bits. The resulting minimum hamming distance is then compared to the threshold; S-boxes with a lower minimum hamming distance are rejected.

If the resulting S-box passes both the differential and linearity tests, it is also tested against the first-order *Bit Independence Criterion* test, to ensure that no pairs of S-box output bits change together more than 50% of the time, when the input changes by a single bit.

The evaluated version of *CRISP* uses S-boxes with a pairs-XOR threshold value of 30, and minimum hamming distance of 0.45215 (926 /2048). The value 30 for the pairs-XOR threshold was chosen because of a currently-uninvestigated runtime complexity phenomenon. When generating random S-boxes in this way, the execution time of the generator increases non-linearly as the threshold value decreases. It was determined that below a threshold value of 30, the program tended towards infinite execution time. Initially, it was thought to be an artifact of the random-number generator in use, so a new one was inserted, with exactly the same result. In any case, the goal of the generator program is to reduce the maximum pairs-XOR count towards the perfect value, which in the case of 11x8 S-boxes is 8 ($2^{11} / 2^8$). The value 30 corresponds to a single-round, single S-box probability of $1.46 \times 10^{-2}$.

The *S()* generation process requires approximately 40 CPU-hours on an HP9000/735.

Examples of S-boxes that correspond to the selection criteria are listed in Appendix B.

## S2() function design

The S2() consists of five 8x32 S-boxes, generated in a key-dependant way. These 8x32 S-boxes are used twice on the 8-bit outputs of the primary S-boxes to produce a 64-bit final result.

The *S2()* function provides an extra stage of confusion and diffusion within the round function. It has the important added benefit of adding to the overall complexity of differential cryptanalysis, reducing the single round, single S-box probability from $1.46 \times 10^{-2}$ to $8.82 \times 10^{-7}$ ($0.0146 * (0.0078)^2$). This comes from recent results[2] on the differential cryptanalysis properties of random 8x32 S-boxes.

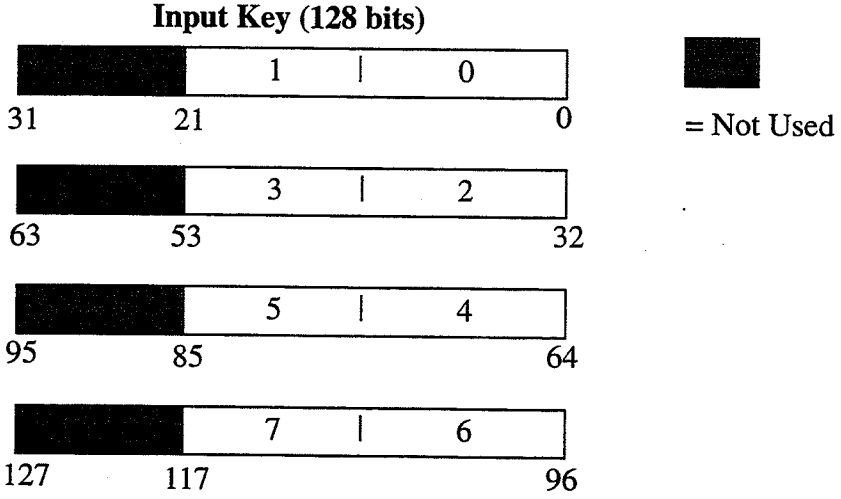
Since the contents of the *S2()* S-boxes are unknown to the cryptanalyst, both linear and differential cryptanalysis are significantly hampered.

## Subkey generation

Subkeys are generated in such a way that if a given subkey is determined by cryptanalysis, it is cryptographically difficult to determine the other subkeys from the known subkey.

This is achieved by using the basic *CRISP* encryption function as a pseudo-random number generator, using the key as a seed. This is accomplished in a multi-step process, described below.

First, a "standard" key-schedule is loaded into the *CRISP* function, the standard key-schedule is derived from the first 48 entries in table 0 and table 7 in the *ES()* function, combined with XOR. This standard key-schedule is then perturbed by selecting bits from the input key, and XOR combining them with the "standard" key schedule, 11 bits at a time. A total of 88 bits from the input key are selected for use in perturbing the "standard" key schedule, as follows:



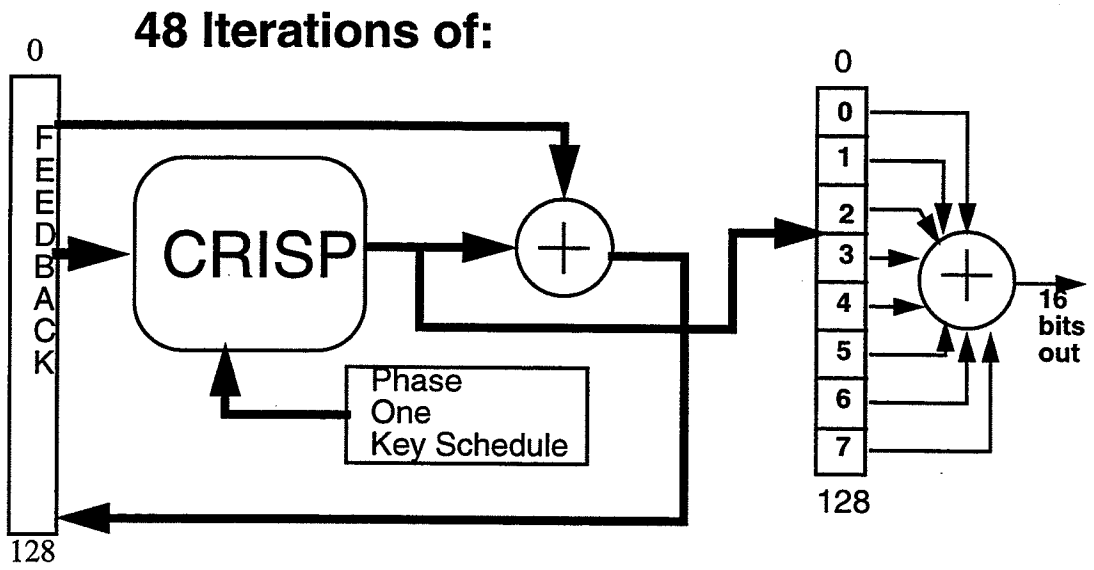

The 48 entries from the ES[0,7] XOR are grouped into six sets of eight elements, producing the following table.

**Table 1: Standard Key Schedule**

| Round | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 1 | 4CC | 079 | 4AA | 7BC | 6C8 | 573 | 3DE | 5EC |
| 2 | 63F | 6EF | 2BF | 1AE | 7F2 | 253 | 595 | 42E |
| 3 | 5E3 | 24B | 7CB | 1D9 | 324 | 341 | 2E6 | 1E2 |
| 4 | 142 | 47C | 26D | 593 | 151 | 028 | 23D | 004 |
| 5 | 527 | 39F | 30C | 217 | 01D | 7A6 | 55B | 1DB |
| 6 | 7FA | 271 | 64E | 4B4 | 316 | 53A | 2B8 | 3A9 |

Each row in this table is XORed with the corresponding (0 through 7) 11-bit value extracted from the key. This slightly-perturbed key-schedule (*phase one* key-schedule) is then used in a feedback execution of *CRISP*, to produce a new key-schedule. The feedback begins by using the key as the initial cleartext, on each iteration, the feedback buffer is updated by XOR with the CRISP ciphertext output. This *phase two* key-schedule is produced by using each output of the feedback execution of *CRISP* to produce 11-bit key-schedule elements that update the *phase one* schedule by one

element on each iteration, for a total of 48 iterations. The diagram below illustrates this concept:




The final key schedule is produced by again using *CRISP* in a feedback mode, with the input key as the initial cleartext, using the *phase two* key schedule, and the standard *S2()* function. Each ciphertext output is considered as eight 16-bit values, each of which is XORed together, then masked down to 11 bits to produce a key-schedule element. This process is repeated until all of the key-schedule elements have been filled. There are eight 11-bit elements per round, with six rounds in the evaluated implementation, for a total of 48 key schedule elements or 528 key schedule bits.

## Generation of the *S2()* function

The *S2()* function is computed in a similar fashion to the final key-schedule, using *CRISP* in feedback mode. This feedback execution is a continuation of the feedback execution used in generating the final key-schedule. Each output of the *CRISP* execution is considered as four 32-bit values. The values are combined using XOR, with the resulting value being placed in the next available *S2()* table element. If the 32-bit value has already been used in an *S2()* table element, it is discarded and a new value is generated.

There are five *S2()* tables, each with 256 entries, for a total of 1280 32-bit elements.

## Comparison of *CRISP* and *DES* round functions

The round function of DES takes a 32-bit input, and computes a non-linear function of that 32-bit input. It accomplishes this using four discrete steps. The 32-bit data input is expanded using the E expansion, then mixed with the 48-bit key-schedule bits. The resulting 48-bit value is then non-linearly substituted using the eight 6x4 S-boxes. The final step is to permute the 32-bit S-box output using the P permutation.

When examining the *E* expansion in DES, notice that it provides no guarantee that a given input bit can affect more than one S-box. This makes differential cryptanalysis easier, since single S-

boxes can be "isolated" for differential cryptanalysis purposes.

The cryptographic significance of the *P* permutation is assumed to be for the purposes of improving the diffusion properties of the round function, since the *E* expansion provides rather less diffusion.

The *CRISP* algorithm has the same basic structure in its round function as DES. The round function takes a 64-bit input, expands it to 88 bits using the ES function, mixes it with the key, and non-linearly substitutes the 88-bits using eight 11x8 S-boxes. When examining the ES function, observe that each input bit affects two S-boxes, thus making differential cryptanalysis somewhat harder. The ES function also provides, as a secondary effect, a small amount of non-linearity, since it acts as a 16x11 S-box. The *CRISP* S-boxes, due to their size, provide higher a degree of resistance both to differential and linear cryptanalysis than DES.

In *CRISP*, the post S-box function, *S2*, corresponds roughly to the *P* permutation in DES. Observe that *S2* provides a non-linear transform of the S-box outputs, while the *P* function in DES is entirely linear. The *S2* function also improves resistance to both differential and linear cryptanalysis, since the *S2* table elements are unknown to the cryptanalyst. Even if the cryptanalyst is able to determine the contents of S2, it is assumed that the analysis of random 8x32 S-boxes, as described in [2], would hold for the *S2* function within *CRISP*.

## Analysis of key-scheduling and *S2()* generation

The strength of the key-scheduling and *S2()* function generation algorithm is predicated on the ability of the concatenation of round functions to act as a random, non-linear transform of the input key material. The avalanche results shown later tend to suggest that *CRISP* does act as a strong random transform, with good per-round non-linearity; the assumption is that the concatenation of rounds produces a non-linearity that is close to the product of the non-linearity of the round function.

The purpose of the key-schedule algorithm is to produce a sequence of bits from the input key material that can be used as per-round keys. Many encryption functions use a key-schedule algorithm in which the round key bits are related to the input key in a way that is linear. The DES, for example, uses a series of rotates and selects to produce round key material. This makes DES slightly more vulnerable to differential cryptanalysis[4] than DES with purely-random, independent round keys. This occurs because determination of one or more per-round key bits results in determination of related bits in other rounds.

It has been proposed, most recently in [5], that the DES can be strengthened somewhat against both linear and differential cryptanalysis by using the DES cipher itself as a PRNG in the generation of key-schedule bits.

The Blowfish[7] cipher also uses itself as a PRNG in the generation of both its S-boxes, and in the generation of the *P()* round function.

An early version of *CRISP* used MD5[6] as a PRNG in the generation of round keys, but it was

felt to be overly complex, and not as compact as a key-scheduling algorithm that uses the *CRISP* cipher itself as a PRNG.

If we assume that the cryptanalyst is able to determine the round key at a particular round, they must be able to determine the plaintexts that correspond to the partial ciphertexts that constitute the determined round key. Each 11-bit round-key element is the XOR of eight 11-bit sections of a *CRISP* ciphertext, under an unknown key, with unknown plaintext. The problem, then, is to determine the full 128-bit ciphertext, and the corresponding plaintext, under an unknown key.

The cryptanalyst has a similar problem to solve when they are able to determine some values within *S2()*. They must first determine, for a given determined 32-bit S-box entry value, the corresponding 128-bit ciphertext, then the key-schedule and plaintext that produced it.

The performance of key-scheduling is entirely dependant on the performance of the *CRISP* algorithm itself. The *CRISP* algorithm is called approximately[1] 1376 times in setting a new key. On the hardware the algorithm was tested on, this corresponds to 20 milliseconds of real time. This could be improved by changing the algorithm that produces *S2()* to produce four *S2()* elements at a time from the full-width 128-bit output of *CRISP*.

## Avalanche Results

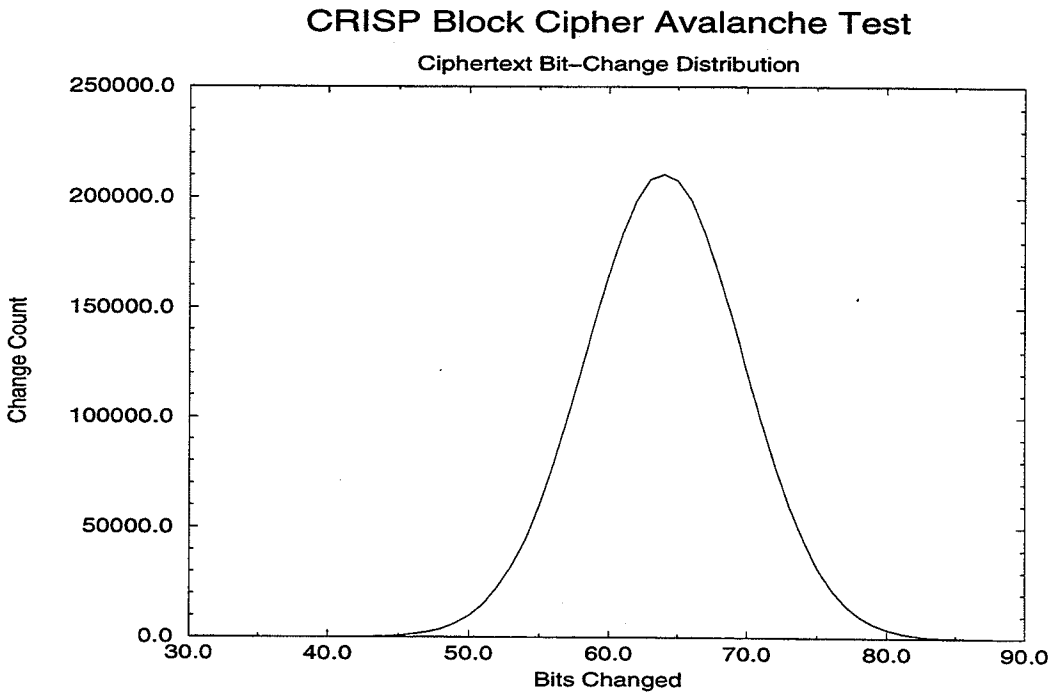
The avalanche properties were measured by iterating the *CRISP* encryption function 3,000,000 times, using a fixed, random key, and a random data input that is modified randomly by one bit on each iteration. This process was repeated several times.

This results in an average change in the resulting ciphertext of 64 bits, which is 50% of the total ciphertext bits. The minimum change ranges from 35 to 37, while the maximum ranges from 85 to 92. The minimum ciphertext change corresponds to somewhat more (0.273 to 0.289) than 25% of the total bits in the ciphertext.

The DES under similar test conditions tends to produce an average ciphertext change of exactly 50% of the bits, while the minimum is usually somewhat less (0.203 to 0.234). The following

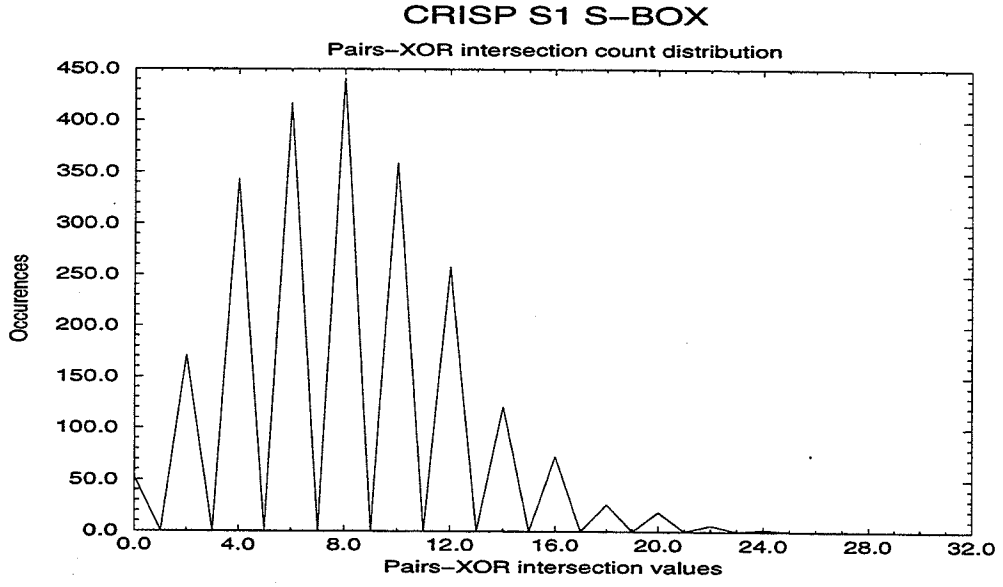
1. Since the *S2()* generation process can potentially call *CRISP* a variable number of times, due to its selection criterion.

graph shows the distribution of ciphertext bit-changes for 3,000,000 iterations of the function:




## Differential Cryptanalysis Results

The pairs-XOR count distribution graph shows that the mean value for a pairs-XOR "intersection" over the entire S1 box from *S()* is 8:



This graph requires some explanation. The usual method to show pairs-XOR distribution uses a table, with the output-XOR as columns, and the input-XOR as rows. The elements of such a table convey the distribution of pairs-XOR values over the given S-box. The *CRISP* S-boxes are 11x8, which means the resulting table would have 256 columns and 2048 rows; values that yield a rather ungainly tabular display. The graph conveys the same overall information, showing that the "intersection" values are clustered around the so-called perfect distribution that would be achieved if each intersection in the pairs-XOR table were equally likely. In the *CRISP* case, that perfect distribution value would be 8 ($2^{11} / 2^8$).

We can observe from the pairs-XOR distribution, that all of the primary S-boxes have the same maximum pairs-XOR value of 30 (or a probability of 0.0146). There is no obvious advantage to attacking a particular S-box over another.

The *ES()* function effectively acts as a fixed 16-to11 bit mapping between bits of the round function input, and input bits to a single S-box. The following table illustrates the mapping:.

**Table 2: ES function input mapping**

| Input Octet Pair | ES box pair | S-box affected |
|---|---|---|
| 5 and 6 | 0 and 1 | S0 |
| 7 and 0 | 2 and 3 | S1 |
| 1 and 2 | 4 and 5 | S2 |
| 3 and 4 | 6 and 7 | S3 |
| 7 and 6 | 6 and 5 | S4 |
| 5 and 4 | 4 and 3 | S5 |
| 3 and 2 | 2 and 1 | S6 |
| 1 and 0 | 0 and 7 | S7 |

In effect, a new set of 16-by-11 bit S-boxes are synthesized by the input mapping to *ES()*.

The problem for the cryptanalyst, then, is to find input octet pairs that produce the maximum differential probability (by minimizing the number of S-boxes involved, and by selecting the highest probability for each S-box involved). In DES, the pre-S-box expansion function, *E*, has the property that for input pair $X_1$ and $X_2$ the equation $E(X_1)\ XOR\ E(X_2) = E(X_1\ XOR\ X_2)$ is always true. In *CRISP*, the equivalent $ES(X_1)\ XOR\ ES(X_2) = ES(X_1\ XOR\ X_2)$ is true for only a small number of input pairs (approximately 1 in 2500). This means that a different approach is required when engaging in differential cryptanalysis. The cryptanalyst must search for input pairs that satisfy the equation $ES(X_1)\ XOR\ ES(X_2) = I$, where $I$ is a desirable input XOR to a target S-box. Because each such input pair controls both the so-called *target* S-box, and partially controls the input to

two other S-boxes, via different *ES()* boxes, it is thought to be difficult to find input pairs that simultaneously satisfy desirable input XOR conditions for the S-boxes they control.

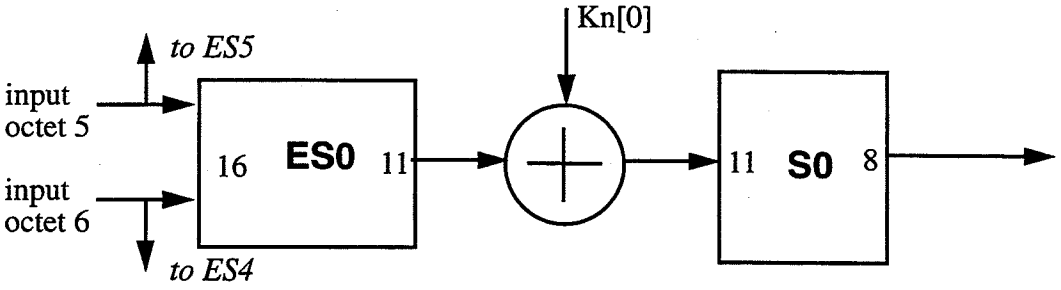
Initial analysis of the density of pairs satisfying the criterion of a high-probability XOR in one S-box, while having a zero XOR in the two other S-boxes linked via common input octets is estimated to be 1 in $2^{28}$, for randomly selected inputs. For example, S-box 0 is linked to S-box 4 and S-box 5 via input octets 5 and 6. This means that the input to these three S-boxes is fully defined by 4 input octets, (octets 4,5,6 and 7). The following input pairs for octets 4,5,6,7 satisfy the above criterion:

$X_1$=7068790E $X_2$=D68D3980
$X_1$=09FC3D28 $X_2$=2898D918
$X_1$=493238F2 $X_2$=CBF8D398
$X_1$=40D72FF9 $X_2$=5465FB57

The above input values do not necessarily guarantee zero input XOR to the other S-boxes affected by input octets 4 and 7 (S-boxes 3 and 1).

It is surmised that inputs satisfying the more stringent criterion of having a high-probability input XOR in one S-box, while having zero XOR in all the others have a very low density. Similarly, inputs satisfying the criterion of having high-probability in two S-boxes, while having zero XOR in all other S-boxes is of a similar density. Initial testing shows that the density may be less than 1 pair in $2^{34}$ random input pairs.

The combined ES() and S() can be re-arranged (with S0 as example), as follows:




Given the above arrangement, we can compute the input XOR distribution for ES() that yields the high-probability input XORs to the corresponding S() S-box. The S0 box, for example, has 12 high-probability inputs XORs (that is input XORs, that have output XORs occurring with probability 0.0146). In S0, a high-probability input XOR is X'6AE'. An input XOR of X'03AA' to ES0 leads to an ES0 output XOR of X'6AE' which in turn leads to an output XOR in S0 of X'17', with compound probability 1.79 x $10^{-5}$. The following table shows examples of the highest com-

pound probabilities of ES0 input XORs producing a given S0 output XOR.

**Table 3: Example ES0/S0 XOR probabilities**

| ES[0] Input XOR | S[0] Output XOR | Probability |
|---|---|---|
| X'054F' | X'E9' | 2.15E-5 |
| X'0DAD' | X'B8' | 2.06E-5 |
| X'6635' | X'97' | 2.06E-5 |
| X'700E' | X'17' | 2.24E-5 |
| X'7B13' | X'97' | 2.15E-5 |
| X'81CB' | X'DA' | 2.06E-5 |
| X'8233' | X'4D' | 2.06E-5 |
| X'FD28' | X'97' | 2.06E-5 |

Since each input octet controls two S-boxes, a reasonable assumption to make is that at least two S-boxes must be "involved" in a given single-round characteristic, thus giving a maximum single-round probability near $4.8 \times 10^{-10}$ (X'700E' to X'17'). If we assume that a six-round characteristic can be constructed in which half the rounds have probability 1, and half the rounds have the probability $4.8 \times 10^{-10}$, then without S2(), *CRISP* is theoretically vulnerable to differential cryptanalysis, since the resulting probability near $1.10 \times 10^{-28}$ is greater than the probability near $2.9 \times 10^{-40}$ that would be necessary to make *CRISP* unconditionally resistant to differential cryptanalysis. If, however, the best characteristic that can be constructed uses the probability of $4.8 \times 10^{-10}$ in all but one round, with a probability 1 characteristic in one round, then *CRISP* would be unconditionally resistant to differential cryptanalysis, since the resulting probability is near $2.5 \times 10^{-47}$. No attempt has yet been made so find the best 6-round characteristic for *CRISP*, since *S2()* is assumed to defeat differential cryptanalysis.

If an *average-case S2()* function is factored into the differential probability analysis, then the algorithm is unconditionally resistant if the best six-round characteristic has probability 1 in three of the rounds, and probability near $2.92 \times 10^{-14}$ in the other rounds, producing an aggregate probability near $2.55 \times 10^{-41}$.

Appendix C contains complete tables of high-probability XORs for the eight ES/S combinations.

## Linear Cryptanalysis Results

Work on linear cryptanalysis is in progress at the time of writing.

## Resistance to the Birthday Paradox under CBC

Under Cipher Block Chaining mode, this cipher is more resistant than ciphers with smaller block

sizes to the Birthday Paradox, which states that, if $2^{n/2}$ plaintexts are encrypted under CBC (where $n$ is the blocksize in bits), the probability of there being two equal ciphertexts is 0.5. If two identical ciphertexts correspond to different plaintexts, then there exists a known XOR relation between the two plaintexts.

Since *CRISP* has a 128-bit block, the probability is vastly less than with 64-bit ciphers.

## Resistance to attacks based on non-surjective round functions

An early version of the algorithm used four S-boxes in *S2()*. This produced a round function that was non-surjective (approximately 40% of the $2^{64}$ outputs were impossible). This lead to the round function being theoretically vulnerable to an attack described by Bart Preneel in [3].

In practice, such an attack is unlikely to succeed, due to the very large tables that must be constructed, on the order of $2^{62.5}$ elements, or approximately $2^{65}$ bytes. Because the round-keys are 88 bits, even with a conservative estimate that the entropy is lower than the 88 bits suggested by the key size, an attack is also unlikely to succeed, even when enough table space is available.

*CRISP* was made substantially more resistant to this attack by the addition of a fifth S-box in the *S2()* function, thus making less than 4% of the $2^{64}$ outputs impossible.

The performance penalty for implementing this was approximately 12.5%, with a 1K byte memory penalty.

## Performance and Memory Requirements

The algorithm was implemented in C, using the GNU-C compiler on an HP-9000/735. The 6-round version produces an encryption rate of approximately 45,000 encryptions per second, or an equivalent data rate of 6.14Mbits per second. Using the native HP/UX compiler produces an approximate 4% performance improvement. There are opportunities for optimization; in particular, the S-box outputs may be composed with the *S2()* function in a single table, reducing the number of table-lookups required in *f()* by 30%.

The tables used to implement *ES()*, *S()*, and *S2()* consume only 25K bytes of memory, which is easily within reach for a microprocessor/embedded controller implementation. The executable code on an HP9000/7XX system is approximately 3K bytes. Implementation complexity can be reduced by changing the key-scheduling algorithm to produce only the key-schedule elements, and dispense with key-dependance in the *S2()* algorithm; the resulting cipher is then potentially subject to standard differential, and linear cryptanalytic attack.

## Acknowledgments

The author gratefully acknowledges the patient tutoring, and expert advice of Carlisle Adams, both in the analysis of the *CRISP* algorithm, and in reviewing early drafts of this paper.

The author would also like to thank his management at *Nortel Technology* for allowing him to pursue topics that are only tangentially related to his main job.

## References

[1] J.A. Gordon, H. Retkin, *Are big S-boxes best?*, Lecture Notes in Computer Science nr. 149, 1983.

[2] J. Lee, H. Heys, S. Tavares, *On the Resistance of the CAST Encryption Algorithm to Differential Cryptanalysis, SAC '95: Workshop Record*, pages 107-119.

[3] Preneel, B., *On Weaknesses of Non-Surjective Round Functions, SAC '95: Workshop Record*, pages 100-106

[4] E. Biham, A. Shamir, *Differential Cryptanalysis of DES-Like Cryptosystems*, Journal of Cryptology, vol.4, 1991, pages 3-72.

[5] U. Blumenthal, S. Bellovin, *A Better Key Schedule for DES-like Ciphers*, paper to appear at PragoCrypt-96, September, 1996.

[6] R. Rivest, *The MD5 Message Digest Algorithm*, published as RFC1321, April 1992.

[7] B. Schneier, *Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish)*, Fast Software Encryption, Cambridge Workshop Proceedings, 1994, pages 191-204.